

URL Shortener Web Application (Advance)

1. Introduction

With the rapid growth of the internet and digital communication, sharing information through web links has become a daily necessity. However, many websites generate long and complex URLs that are difficult to remember, copy, and share. Long URLs can also appear unprofessional and inconvenient when shared through emails, messages, or social media platforms.

A URL Shortener Web Application addresses this issue by converting long URLs into shorter, easy-to-share links. When a user accesses the shortened URL, they are automatically redirected to the original long URL. Apart from shortening links, modern URL shorteners also provide additional features such as user authentication, link history, and data persistence.

This project aims to strengthen understanding of backend development concepts such as authentication, session management, database relationships, and frontend–backend interaction.

2. Technology Stack

The following technologies were used in the development of this project:

Flask: Flask is a lightweight Python web framework that provides tools for routing, request handling, and template rendering. It is easy to learn and highly flexible, making it ideal for educational projects and small to medium-scale applications. In this project, Flask is responsible for handling user requests, managing routes, and coordinating communication between the frontend and backend.

SQLAlchemy ORM: SQLAlchemy is an Object Relational Mapping (ORM) library that allows interaction with relational databases using Python classes instead of raw SQL queries. ORM improves code readability, maintainability, and security. In this application, SQLAlchemy is used to define database models for users and URLs, manage database sessions, and perform CRUD operations efficiently.

SQLite: SQLite is a lightweight, file-based database system that does not require a separate server. It is suitable for small and medium-scale applications. SQLite is used in this project to store user details and shortened URLs persistently.

Bootstrap: Bootstrap is a popular frontend framework that helps create responsive and visually appealing web pages. It provides predefined CSS classes and components such as buttons, forms, and layouts, which enhance the user interface without extensive custom styling.

3. System Architecture

The URL Shortener application follows a three-tier architecture consisting of the frontend, backend, and database layers.

Frontend Layer

The frontend is developed using HTML, CSS, and Bootstrap. It consists of three main web pages:

- Login Page: Allows existing users to log in using their credentials.
- Signup Page: Allows new users to create an account by providing a unique username and password.
- Dashboard Page: Displays the URL shortener interface where logged-in users can shorten URLs, copy shortened links, and view their URL history.

Backend Layer

The backend is implemented using Flask. It handles:

- User authentication and session management
- Username validation and password hashing
- URL validation and short code generation
- Redirecting shortened URLs to original URLs
- Fetching user-specific URL history

Flask routes act as controllers that process incoming requests and send responses to the frontend.

Database Layer

The database layer uses SQLite along with SQLAlchemy ORM. The database stores:

- User credentials (securely hashed passwords)
- Original URLs
- Generated short codes
- User–URL relationships

This separation of layers ensures scalability, security, and maintainability.

4. User Authentication and Validation

Signup Validation

During signup, the application enforces the following rules:

- Username must be unique
- Username length must be between 5 and 9 characters
- Password is securely stored using hashing

If a user attempts to register with an existing username, an error message is displayed. This ensures data consistency and avoids conflicts.

Login Authentication

During login, the entered username and password are verified against stored credentials. Passwords are compared using secure hashing techniques instead of plain text comparison. Upon successful login, a session is created to track the user's activity.

Session Management

Flask sessions are used to maintain user login state. Only authenticated users can access the dashboard and URL-shortening features.

5. URL Validation and Shortening Logic

URL validation is an important part of the application. Users may enter invalid or malformed URLs, which should not be processed. To prevent this, the validators Python library is used to verify whether a URL is valid.

Short Code Generation

Once a valid URL is entered:

1. A random alphanumeric short code is generated.
2. The original URL and short code are stored in the database.
3. The shortened URL is displayed to the user.

This ensures that each shortened URL is unique and reliable.

6. Database Design

The database design consists of two main tables:

6.1 User Table

Column	Description
id	Primary key
username	Unique username
password	Hashed password

6.2 URL Table

Column	Description
id	Primary key
original_url	Original long URL
short_code	Generated short code
user_id	Foreign key referencing User

Using SQLAlchemy ORM, relationships are established between users and their URLs, ensuring that each user can access only their own URL history.

7. Challenges Faced

Username Validation: Ensuring uniqueness and correct length of usernames required careful validation logic and database checks.

Secure Password Storage: Storing passwords securely was achieved using hashing techniques provided by the Werkzeug library.

URL Validation: Handling invalid URLs was challenging and was solved by integrating the validators library.

Database Integration: Integrating Flask with SQLAlchemy and managing application context required careful configuration.

8. Conclusion and Future Enhancements

The Advanced URL Shortener Web Application successfully fulfills all the project objectives. It provides a secure and user-friendly platform for shortening URLs, managing user accounts, and storing link history.

Through this project, key backend development concepts such as authentication, ORM usage, session handling, and validation were learned and implemented.

Future Enhancements

- Click analytics for shortened URLs
- Custom short URLs
- Link expiration feature
- Deployment on cloud platforms
- Improved UI animations

Overall, this project provided valuable hands-on experience in full-stack web development using Flask and strengthened backend programming skills.